

The Complete Rules for AI-Generated Apps

Security, Engineering, and Product Checklist

Security checklist by Taha Jaffri | Expanded with engineering & product analysis

23 rules + AI-specific checks, covering both security and the engineering, product, and operational issues that AI coding tools quietly skip. Drop this in your project root as a CLAUDE.md or .cursorrules reference. Every app generated in that project then has a standard to follow — no exceptions, no prompting it twice.

AI tools can write code fast. They do not care if your app is secure, scalable, maintainable, legally compliant, or operationally sound. The output looks right, it runs, and it gets shipped. Everything else follows quietly behind it.

This guide is split into two parts: Part 1 covers the thirteen security categories most commonly missed by AI-generated code. Part 2 covers ten categories of engineering, product, and operational issues that are not security risks on their own, but will still break, slow down, or sink a vibe-coded app. Each rule includes the checklist to follow and, where relevant, a drop-in prompt or code pattern to enforce it.

PART 1 — SECURITY RULES

Thirteen categories. Every one covers a real-world vulnerability that AI-generated apps miss regularly. Work through each section and use the Quick Reference table at the end before every deploy.

1. Secrets and Environment Variables

API keys hardcoded in frontend files are one of the most common ways apps get breached. AI tools do it constantly unless told otherwise.

Rule

Never expose secrets in frontend code

- Every API key, token, database URL, and private config lives in .env files only.
- .env files must always be listed in .gitignore (.env, .env.local, .env.*.local).
- Frontend code must never contain raw secret values — no `const API_KEY = "sk-..."`.
- For Next.js/Vite: only `NEXT_PUBLIC_` / `VITE_` prefixed vars belong in the frontend, and never secret keys.
- Generate a .env.example file with variable names but empty values.

```
// Correct
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

// Wrong – never do this
const stripe = require('stripe')('sk_live_abc123...');
```

2. Rate Limiting

Every public-facing endpoint needs rate limiting, or someone can hammer your auth routes, run up your AI costs, or flood your upload endpoint.

Rule

Apply rate limiting on all API routes

- Auth endpoints (login, register, reset): 5 requests / 15 min / IP.
- General API: 60 requests / minute / IP.
- AI/LLM proxy endpoints: 10 requests / minute / user.
- File uploads: 5 requests / minute / IP.
- Always return 429 with a Retry-After header; never silently swallow it on the frontend.

```
// Express rate limiting
import rateLimit from 'express-rate-limit';
const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 100 });
app.use('/api/', limiter);
```

3. Input Validation and Sanitization

Never trust user input. Client-side validation is a UX convenience, not a security measure — everything that matters happens on the server.

Rule

Validate and sanitize everything on the server

- Use schema validation: Zod/Joi (JS/TS), Pydantic (Python).
- Sanitize all string inputs before storing or displaying to prevent XSS.
- Use parameterized queries or ORM methods — never interpolate input into raw queries.
- Validate type, length, allowed characters, required fields, enum values.
- For file uploads, validate MIME type, extension, and size server-side.

```
import { z } from 'zod';
const schema = z.object({
  email: z.string().email().max(254),
  message: z.string().min(1).max(1000).trim(),
});
const result = schema.safeParse(req.body);
if (!result.success) return res.status(400).json({ error: result.error });
```

4. Authentication and Authorization

Auth is where most apps fail. Rolling your own from scratch is almost always a mistake.

Rule

Use established auth libraries and follow password rules

- Use NextAuth.js, Clerk, Supabase Auth, Auth0, Passport.js, or Lucia-auth.
- Passwords: never plain text — bcrypt (min cost 12) or argon2.
- JWTs signed with a strong secret (32+ chars), short expiry (15–60 min).
- Refresh tokens in httpOnly cookies, not localStorage.
- Verify identity AND permission on every request; add explicit role checks on admin routes.

```
// Always check ownership, not just authentication
const post = await db.post.findUnique({ where: { id } });
if (!post || post.authorId !== session.user.id) {
  return res.status(403).json({ error: 'Forbidden' });
}
```

5. SQL and Database Security

SQL injection still works because AI-generated code will happily concatenate user input into a query if you let it.

Rule

Always use an ORM or parameterized queries

- Use Prisma, Drizzle, SQLAlchemy, or Mongoose.
- Never construct queries via string concatenation with user data.
- Apply least privilege: the DB user only has the permissions it needs.
- Never return raw database errors to the client — they leak schema info.

```
// Safe parameterized query
const user = await db.query('SELECT * FROM users WHERE email = $1', [email]);

// Never do this
const user = await db.query(`SELECT * FROM users WHERE email = '${email}'`);
```

6. CORS Configuration

Wildcard CORS in production means any website can make requests to your API on behalf of your users.

Rule

Never use wildcard CORS in production

- Explicitly whitelist only origins that should access your API.
- Restrict allowed HTTP methods to only what each endpoint needs.
- Use the credentials flag only when actually required.

```
app.use(cors({
  origin: process.env.ALLOWED_ORIGIN,
  methods: ['GET', 'POST'],
  credentials: true,
```

```
}});
```

7. HTTP Security Headers

Security headers are free protection. Most apps ship without them, and a few lines of config can stop entire categories of attacks.

Rule

Always set security headers using helmet or equivalent

- Content-Security-Policy: restrict script/style sources.
- X-Frame-Options: DENY to prevent clickjacking.
- X-Content-Type-Options: nosniff.
- Strict-Transport-Security to force HTTPS.
- Remove X-Powered-By to avoid leaking framework information.

8. File Upload Security

File uploads are a common attack surface. Validate everything on the server and never trust what the client says about a file.

Rule

Validate, rename, and store uploads safely

- Validate MIME type and extension server-side, never trust the client.
- Strict size limits: 5MB images, 25MB documents as a default.
- Store files outside the web root or in S3/GCS/Cloudinary.
- Never serve uploads with executable permissions.
- Rename uploads to a UUID — never use the original filename.

9. Error Handling and Logging

Stack traces in production responses are a gift to attackers — they reveal exactly where your code lives and what its weak points are.

Rule

Never return internal errors to the client

- Return generic messages to users ("Something went wrong" is enough).

- Log errors server-side with full context: timestamp, user ID, route, sanitized input.
- Use Sentry, Datadog, or Logtail in production.
- Use correct status codes — never 500 for a validation failure.

10. Dependency Security

Third-party packages are code you didn't write, running in your app. Audit them and don't install what you haven't thought about.

Rule

Audit dependencies and pin versions in production

- Run npm audit / pip-audit after installing packages; fix high/critical issues.
- Avoid unmaintained packages (no updates in 2+ years for security-relevant libs).
- Pin versions via package-lock.json / requirements.txt.
- Review packages with excessive permissions or suspicious install scripts.

11. XSS Prevention

Cross-site scripting happens when untrusted content is rendered as HTML. AI-generated React code often reaches for dangerouslySetInnerHTML without thinking.

Rule

Never render dynamic user content as raw HTML

- Avoid dangerouslySetInnerHTML unless content is sanitized with DOMPurify.
- Never use eval(), new Function(), or innerHTML with dynamic content.
- Avoid inline script tags — keep JS external to enable CSP enforcement.

12. Deployment Checklist

Before every deploy, run through this list. It takes two minutes and catches what slips through at the end of a build.

Rule

Pre-deploy gate

- .env is not committed to git.
- All secrets set in the hosting platform's env config.

- Debug mode and dev logging are off in production.
- Database is not publicly exposed; HTTPS is enforced.
- Rate limiting active on all public endpoints; CORS restricted to known origins.
- Unused API routes are removed or protected.

13. AI and LLM-Specific Rules

If your app uses an AI API, there's an extra layer of concerns: prompt injection, cost attacks, and insecure rendering are all real problems in production AI apps.

Rule

Treat LLM inputs and outputs like untrusted data

- Never send raw user input directly to an LLM — sanitize it first against prompt injection.
- Always set a max_tokens limit on LLM calls to prevent runaway costs.
- Store the API key server-side only; route all LLM calls through your backend, never the browser.
- Log LLM usage (token counts) per user to detect abuse early.
- Implement per-user/session token budgets.
- Validate and sanitize LLM output before rendering it in the UI — generated HTML is an XSS risk.

PART 2 — ENGINEERING & PRODUCT RULES

These ten categories will not show up in a security audit, but they are exactly what tends to quietly break, slow down, or sink a vibe-coded app after launch: architecture, scale, cost, reliability, data integrity, UX, maintainability, legal exposure, operations, and the AI's own blind spots.

14. Architecture and Code Structure

AI tools append code wherever the cursor is. Without explicit structure, business logic, data access, and UI rendering all end up tangled in one file.

Rule

Enforce separation of concerns and consistent patterns

- Separate routes/handlers (orchestration) from business logic (services) from data access (repositories/ORM).
- Enforce one pattern per concern across the whole codebase — don't mix async/await with raw callbacks.
- No direct DB or API calls inside UI components; route them through a service/hook layer.
- Reuse existing functions instead of regenerating similar logic.
- Flag and refactor any function over ~50 lines or file with more than one responsibility.

Drop-in prompt for CLAUDE.md / .cursorrules

*Before writing new code, check the existing codebase structure. Maintain this layering:
UI components -> hooks/controllers -> services (business logic) -> repositories/ORM (data access).*

Never call the database or an external API directly from a UI component.

Reuse existing functions/services instead of duplicating logic. Use one consistent async pattern (async/await) throughout the project.

15. Scalability and Performance

Works fine with 50 test rows, falls over at 50,000. AI-generated code is rarely written with real-world scale in mind.

Rule

Design for more than the demo dataset

- Avoid N+1 queries — batch or join instead of looping individual queries.
- Add indexes on any field used in WHERE, JOIN, or ORDER BY clauses.

- Paginate any endpoint that can return more than ~100 records.
- Cache expensive or repeated computations and API calls.
- Clean up listeners, intervals, and subscriptions on component teardown to avoid memory leaks.
- Avoid blocking synchronous operations in async runtimes.

Drop-in prompt for CLAUDE.md / .cursorrules

Assume this app will run with 100x more data and users than the current test data. Avoid N+1 queries -- batch or join instead of looping individual queries. Add pagination to any endpoint that could return more than 100 records. Add database indexes on fields used in filtering or sorting. Always clean up timers, intervals, and event listeners in component unmount/cleanup logic.

16. Cost and Resource Management

Unbounded LLM calls, oversized cloud instances, and forgotten cron jobs are how a side project turns into a surprise invoice.

Rule

Treat compute and API calls as money, not abstractions

- Cache identical or duplicate LLM/third-party API requests instead of recalling them.
- Set hard per-user and per-session quotas on paid API calls.
- Add monitoring/alerts for unusual spend or usage spikes.
- Right-size cloud resources to actual load — don't default to the largest instance.
- Tear down or schedule-stop unused dev/staging environments, cron jobs, and storage buckets.

Drop-in prompt for CLAUDE.md / .cursorrules

Treat every external API call (LLM, email, SMS, storage) as a billed action. Cache repeated identical requests. Add per-user usage limits on top of basic rate limiting. Log usage per user so spend can be tracked and capped. Flag any code that calls a paid API in a loop without a cap.

17. Testing and Reliability

AI-generated code rarely comes with tests unless explicitly asked for, and silent failures hide bugs until a user finds them in production.

Rule**Ship with a safety net, not blind faith**

- Write unit tests for core business logic, especially anything touching money, auth, or data mutation.
- Add error boundaries (React) or global exception handlers (backend) so one failure doesn't crash everything.
- Handle and test edge cases explicitly: empty states, network failure, concurrent requests.
- Run tests and lint checks in CI before merge/deploy, not after.
- Never let a try/catch silently swallow an error — log it or surface it.

Drop-in prompt for CLAUDE.md / .cursorrules

For any function that touches payments, authentication, or data mutation, generate accompanying unit tests covering the happy path, at least one failure path, and one edge case (empty/null/concurrent). Never leave a catch block empty -- log the error or rethrow it. Add a CI step that runs tests and linting before any deploy.

18. Data Integrity and Business Logic

Security stops attackers; data integrity stops your own app from corrupting itself when two requests collide or a step in a process fails halfway through.

Rule**Protect the data, not just the perimeter**

- Wrap multi-step operations (charge -> create record -> update inventory) in a database transaction.
- Add business-level validation beyond format checks (no negative prices, no orphaned foreign keys).
- Use database-level constraints (NOT NULL, UNIQUE, foreign keys), not just app-level checks.
- Handle race conditions explicitly with locks, optimistic concurrency, or atomic operations.
- Maintain an audit log for sensitive state changes.

Drop-in prompt for CLAUDE.md / .cursorrules

Wrap any multi-step write operation (e.g. payment + order creation + inventory update) in a single database transaction so partial failures cannot leave inconsistent state. Add database-level constraints (NOT NULL, UNIQUE, foreign keys), not just app-level validation.

For any shared resource updated by multiple requests, use locking or atomic operations to prevent race conditions.

19. UX, Accessibility, and Consistency

Each screen generated in isolation tends to look and behave slightly differently — buttons, modals, and error states drift apart fast.

Rule

Design once, then reuse it everywhere

- Define and reuse a single design system/component library instead of regenerating UI per screen.
- Every screen needs a loading state, an empty state, and an error state — never a blank screen.
- Use semantic HTML, proper labels, and keyboard navigation; test with a screen reader once before shipping.
- Test layouts on at least one real mobile viewport, not just the AI's assumed desktop size.
- Keep onboarding and error copy in plain language for non-technical users.

Drop-in prompt for CLAUDE.md / .cursorrules

Reuse existing UI components instead of generating new ones for each screen. Every data-fetching component must handle three states: loading, empty, and error -- never leave a blank screen. Use semantic HTML elements and accessible labels (aria-label, alt text) by default. Assume the user is on a mobile viewport unless told otherwise.

20. Maintainability and Technical Debt

If you can't explain why a piece of AI-generated code works, neither can anyone you hand it to six months from now.

Rule

Write it so someone else (or future you) can read it

- Every project needs a README covering setup, environment variables, and how to run it locally.
- Use descriptive names for variables, functions, and files — no generic temp, data2, handleStuff.
- Comment the "why," not the "what."

- Pin dependency versions and review what changed before upgrading.
- Keep dev, staging, and prod configs in version-controlled templates so they don't silently diverge.

Drop-in prompt for CLAUDE.md / .cursorsrules

Generate or update a README whenever a new setup step, environment variable, or script is introduced. Use descriptive, specific names -- never generic placeholders like data, temp, or handleStuff. Add comments only to explain non-obvious decisions, not to restate what the code already says.

21. Legal, Compliance, and Privacy

Collecting user data without a policy, a deletion flow, or a license check is a legal liability that has nothing to do with code quality.

Rule

Treat user data like someone else's property

- Add a privacy policy and terms of service before collecting any personal data, even at MVP stage.
- Build a data deletion/export flow for stored personal data (GDPR/CCPA-style requirement).
- Check the license of every dependency before commercial use — GPL-family licenses can require open-sourcing your own code.
- Keep an audit trail for actions on sensitive data (admin access, payment changes, data exports).
- Don't collect data you don't have a clear use for.

Drop-in prompt for CLAUDE.md / .cursorsrules

Before adding any feature that collects personal data, confirm there is a privacy policy and a way for the user to request deletion of their data. Check the license of any new dependency before installing it and flag GPL/AGPL-licensed packages for review before use in a commercial app.

22. Operational Readiness

No monitoring, no backups, no rollback plan — most vibe-coded apps find out about all three at the same time, during an outage.

Rule**Plan for the day something breaks**

- Set up uptime and error monitoring (Sentry, UptimeRobot, Datadog) before launch, not after the first outage.
- Take automated, regular database backups, and actually test restoring from one.
- Have a documented rollback procedure for a bad deploy (previous build, previous migration).
- Avoid single points of failure where feasible; at minimum, know what fails first and the recovery time.

Drop-in prompt for CLAUDE.md / .cursorrules

Before marking a feature as production-ready, confirm: error monitoring is wired up, the database has automated backups, and there's a documented way to roll back this deploy if it fails. Do not consider a launch complete without these three in place.

23. AI-Specific Engineering Pitfalls

AI-generated code can look entirely plausible and still call a method that doesn't exist, or use a pattern that was deprecated two years ago.

Rule**Don't trust the AI's code more than you'd trust a junior dev's**

- Verify that any library function, API method, or config option actually exists and behaves as described.
- Check that suggested libraries/patterns aren't deprecated.
- Review AI-generated code for unnecessary complexity or missing edge cases — both happen depending on how the prompt was framed.
- Re-read AI-generated error messages and UI copy for tone consistency with the rest of the app.

Drop-in prompt for CLAUDE.md / .cursorrules

After generating code that uses a third-party library or API, verify the method names and parameters against the current official documentation before trusting it. Flag any usage of a deprecated package or pattern. Review generated code once for complexity -- simplify if it solves a simpler problem than it appears to, and add handling if it's missing obvious edge cases.

Quick Reference — All 23 Areas

One table. Everything in the guide, security and engineering combined. Bookmark this page and run through it before every ship.

Area	Core Rule	Tools / Examples
Secrets	Keys in .env only. Never in frontend code.	.gitignore, .env.example
Rate Limiting	5 req/15 min auth. 60 req/min general API.	express-rate-limit, slowapi
Input Validation	Server-side only. Schema validation required.	Zod, Pydantic
Auth	bcrypt min cost 12. JWT short expiry. httpOnly cookies.	NextAuth, Clerk, lucia-auth
SQL Security	ORM or parameterized queries only. No string concat.	Prisma, Drizzle, SQLAlchemy
CORS	No wildcard in production. Explicit origin whitelist.	cors() with origin config
HTTP Headers	CSP, HSTS, X-Frame-Options: DENY.	helmet (Node), django-csp
File Uploads	MIME + extension validation. UUID rename. Outside web root.	multer, S3, Cloudinary
Error Handling	Generic messages to client. Full context in logs.	Sentry, Datadog, Logtail
Dependencies	Audit after every install. Pin versions in production.	npm audit, pip-audit
XSS	No dangerouslySetInnerHTML. No eval(). No inline scripts.	DOMPurify
Deploy Gate	Run checklist before every ship.	See Section 12
AI / LLM Security	Sanitize input. Server-side keys. Token budgets.	Server proxy, max_tokens
Architecture	Layered structure. One pattern per concern.	Services, repositories
Scalability	Design for 100x data. No N+1, paginate, index, cache.	Redis, DB indexes
Cost Management	Cap paid API usage per user. Monitor spend.	Usage logs, billing alerts
Testing	Unit tests on critical paths. CI before deploy.	Jest, Pytest, CI pipeline
Data Integrity	Transactions for multi-step writes. DB constraints.	Prisma transactions
UX / Accessibility	Loading/empty/error states. Semantic HTML.	Design system, ARIA
Maintainability	README, descriptive naming, pinned versions.	README.md, lockfiles
Legal / Privacy	Privacy policy. Data deletion flow. License checks.	GDPR/CCPA basics
Operational Readiness	Monitoring, backups, rollback plan before launch.	Sentry, automated backups
AI Code Reliability	Verify AI-used APIs exist. Check for deprecation.	Official docs review

Based on the security checklist by @tahajaffrii — expanded with engineering, product, and operational analysis.